

1 Conceptual: Technical Debt Diagnosis

- Entanglement - Changing Anything Changes Everything (CACE)
 - Undeclared Consumers
 - Configuration and Glue-Code Debt
- The glue-code debt can be mitigated by writing clean modular code and all hyperparameters decoupled from the script(s) and organized in a YAML file, preferably tracked using MLFlow or wandb.

2 Applied: MLFlow Experiment Comparison

activation	logistic	logistic	logistic	relu	relu	relu
learning_rate	0.1	0.01	0.001	0.1	0.01	0.001
Metrics						
Show diff only						
Train accuracy	0.56	0.888	0.962	0.1	0.691	0.988
Train f1_macro	0.537	0.867	0.962	0.018	0.684	0.987
Train loss	1.312	0.382	0.138	2.312	0.925	0.042
Val accuracy	0.559	0.885	0.954	0.099	0.688	0.96
Val f1_macro	0.539	0.883	0.954	0.018	0.68	0.959
Val loss	1.219	0.387	0.155	2.308	1.003	0.407

-
- The best performing run solely based on validation metrics was using a learning rate of 0.001 and ReLU activation, achieving an accuracy of 96% and F1 score of 0.959 on the validation data.

However, there is some evidence of **overfitting** in the case of **learning rate of 0.001 with ReLU activation**, as the training accuracy hits 98.8% but validation accuracy remains 96%, with the train-val loss gap being 0.36, which is relatively very high compared to other architectures.

Hence, the better alternative would be to select **learning rate of 0.001 with Sigmoid activation**, which achieves a validation accuracy of 95.4% (very much comparable to the 96% achieved by ReLU model) and validation F1 score of 0.954, but with a lower train-val loss gap and the lowest validation loss (0.155) among all architectures.

Learning rate seems to have the highest influence on performance, while activation function does not change the performance much.

- with `mlflow.start_run(run_name=run_name):`

```
mlflow.log_params({"learning_rate": learning_rate, "activation": activation})
model, train_loss, train_acc, train_f1, val_loss, val_acc, val_f1 = \
train_and_evaluate(learning_rate=learning_rate, activation=activation)
mlflow.log_metrics({"Train loss": train_loss, "Train accuracy": train_acc,
"Train f1_macro": train_f1})
mlflow.log_metrics({"Val loss": val_loss, "Val accuracy": val_acc,
"Val f1_macro": val_f1})
mlflow.sklearn.log_model(model, name="model")
```

3 Applied: DVC Data Versioning & Rollback

```
(venv) pakshal@Laptop:~/lab/assn1$ wc filenames.csv
1801 1801 54993 filenames.csv
```

```
(venv) pakshal@Laptop:~/lab/assn1$ wc filenames.csv
1801 1801 54993 filenames.csv
```

`filenames.csv` was successfully restored using `git checkout` and `dvc checkout`. Refer to the video for more details.

4 Capstone: End-to-End Reproducibility Drill

Refer <https://github.com/Pakshal-Nagda/AIOps-Lab-Assignment-1-Q4>